

Chapter 43: Internal Architecture

1. Overview

Every Angular developer learns the outer API surface — components, directives, services, templates, change detection triggers. Almost none of them are ever forced to look underneath it, because the framework is designed so you don't have to. But at staff/principal level, "Angular works" is not an acceptable mental model. You need to be able to answer: *what does a running Angular application actually look like in memory, right now, at this line of code?*

The answer is: **a tree of arrays**.

Since Ivy (Angular 9+), Angular is built on two cooperating halves:

- **The compiler** (`@angular/compiler`, `@angular/compiler-cli`), which takes your templates and decorators and turns them into plain TypeScript/JavaScript — specifically, into calls to a small, stable **instruction set** (functions like `ɵɵelementStart`, `ɵɵtext`, `ɵɵproperty`, `ɵɵlistener`). This happens at build time (AOT) or, historically, at bootstrap time (JIT).
- **The runtime** (`@angular/core`), a relatively small library that *executes* those instructions. The runtime doesn't know anything about your specific template syntax — it only knows how to run instruction calls against two core data structures: **LView** (the per-instance, mutable, "live" state) and **TView** (the per-type, static, shared "template" metadata).

This compiler/runtime split is the single most important architectural fact about modern Angular. It is what makes Ivy "locality principle" compilation possible (each component compiles independently, without whole-program knowledge), what makes tree-shaking of unused directives/pipes possible, and what makes the framework's rendering model swappable — the same compiled instructions can paint to the DOM (`platform-browser`), to a virtual DOM-like string buffer for SSR (`platform-server`), or to nothing at all (testing, NativeScript-style renderers) — because the instructions never touch the DOM directly. They call through a `Renderer2 / Renderer3` abstraction.

This chapter builds the mental model from the ground up: what **LView** and **TView** actually contain, how the instruction-set model turns a template into imperative array operations, how the renderer abstraction decouples "what to render" from "where to render it," and how NgModules and standalone components both ultimately compile down to the same runtime primitives (`ɵcmp`, `ɵmod`, `ɵinj` static properties).

2. Core Concepts

2.1 The compiler/runtime split, precisely

Before Ivy (View Engine), the compiler generated a large amount of per-component *factory/rendering code* — literally a JS function per component that imperatively created and updated DOM. This meant compiled output was large, tightly coupled to whichever other components/modules were compiled together (whole-module compilation), and not tree-shakeable.

Ivy inverted this. The compiler now emits:

1. **Static metadata** attached to the class as `ɵcmp` (`ɵɵdefineComponent`), `ɵdir`, `ɵpipe`, `ɵmod`, `ɵinj`, `ɵfac`.

2. A **template function** — a single function per component containing calls into a fixed, versioned instruction set exported by `@angular/core` (`ɵɵelementStart`, `ɵɵelementEnd`, `ɵɵtext`, `ɵɵtextInterpolate1`, `ɵɵproperty`, `ɵɵlistener`, `ɵɵadvance`, etc.).

The runtime (`@angular/core`) ships the *implementations* of those instructions. Because the instructions are a small, stable, versioned contract, the compiler and runtime can evolve independently, and compiled libraries (`.d.ts` + emitted JS with `ɵɵ` metadata) remain usable across compatible runtime versions. This is also why Ivy compilation is "local": compiling `AppComponent` doesn't need to see the internals of every component it uses — it only needs their public compiled shape (selector, inputs, outputs, exportAs, template dependencies) to emit correct instruction calls.

2.2 TView — the static template blueprint

`TView` ("Template View") is created **once per component type** (well, once per distinct template — see the node-reuse/shared TView note below) and cached on the component definition. It holds everything the runtime can precompute and share across every instance of that component, so per-instance work at creation/update time is minimized.

Conceptually, a `TView` contains:

- `data`: a parallel array to the `LView`'s data array, but holding *static per-node metadata* instead of live values — one `TNode` per element/text/container/etc., plus slots for directive defs, pipe defs, etc.
- `firstCreatePass`: a boolean flag — `true` only during the very first time this `TView` is instantiated. Many instruction functions branch on this flag: they do expensive structural work (building `TNode`s, resolving directive matches, allocating slots) only on the first pass, and skip straight to cheap update-only work on every subsequent instantiation of a new component instance.
- `bindingStartIndex`: the index in the flattened data array where the "static creation" section ends and the "bindings" section begins.
- `expandoStartIndex` / **host binding metadata**: bookkeeping for host bindings and directive-contributed bindings that get "expanded" into the `LView` layout.
- `template`: reference to the compiled template render function itself.
- `viewQuery` / `contentQueries`: compiled query-execution functions.
- `components`: indices of child component nodes that need their own change-detection pass triggered.
- `cleanup` / `contentHooks` / `viewHooks` / `destroyHooks`: arrays describing what lifecycle/cleanup work must run and at which index.
- `schemas`, `consts`: compiled constants (e.g., static attribute arrays) referenced by index from instructions to avoid re-allocating literals per instance.

The critical insight: `TView` **never changes per instance**. Two `<app-item>` components on screen share the exact same `TView` object. All the "is this node a component, what directives match here, what's the static attribute list" work is computed exactly once and then reused, which is why Angular's change detection is comparatively fast even with instance counts scaling to the thousands.

There's also a category split you should know cold: `TView.type` distinguishes between a **Root** `TView` (host view of the bootstrapped root), **Component** `TViews`, and **Embedded** `TViews` (for `*ngIf` / `*ngFor` / `<ng-template>` structural view instances) — embedded views reuse the `TView` of their defining `<ng-template>` across every instantiation (e.g. every `*ngFor` iteration shares one `TView`), which is exactly why `*ngFor` is

cheap to re-instantiate: only the LView is fresh per iteration, the TView is shared.

2.3 LView — the live, per-instance state

LView ("Logical View" / "Live View") is the mutable, per-instance twin of TView. If TView is "the class," LView is "the object." Every component instance, and every embedded view instance (every *ngFor row, every *ngIf branch instantiation), gets its own LView.

An LView is, at heart, a JS array (not a class instance — this is a deliberate micro-optimization: array element access is faster than property access with polymorphic shapes, and V8 optimizes monomorphic arrays well). It is indexed by well-known constant slots followed by a flat, growing region of "data."

Fixed header slots (defined as constants in @angular/core's interfaces/view.ts, roughly):

Index	Slot	Meaning
0	TVIEW	back-pointer to this view's TView
1	FLAGS	bitmask: dirty, attached, creation-mode, etc.
2	PARENT	parent LView
3	NEXT	next sibling view in a view-container linked list
4	TRANSPLANTED_VIEWS_TO_REFRESH	count for views moved across containers (ngTemplateOutlet, portals)
5	T_HOST	the TNode that hosts this view (e.g., the component's own element node)
6	CLEANUP	array of cleanup functions/contexts (listener unregistration, etc.)
7	CONTEXT	the component instance / embedded view context object
8	INJECTOR	the LView's node injector data
9	ENVIRONMENT	LViewEnvironment — renderer factory, sanitizer, etc.
10	RENDERER	the Renderer2 / Renderer3 instance used to create/mutate DOM for this view
11	CHILD_HEAD / CHILD_TAIL	linked list of child views (embedded views / component views nested inside)

Index	Slot	Meaning
...	DECLARATION_VIEW, DECLARATION_COMPONENT_VIEW, DECLARATION_LCONTAINER	needed to resolve <code><ng-template></code> contexts and DI across projection boundaries
...	QUERIES, ID, EMBEDDED_VIEW_INJECTOR, ON_DESTROY_HOOKS, EFFECTS_TO_SCHEDULE, REACTIVE_TEMPLATE_CONSUMER	signals/effects bookkeeping, dev-mode ids, etc.

`HEADER_OFFSET` (currently `20` in modern Angular, it has crept up slightly across versions as more bookkeeping slots were added — the exact number is an implementation detail you should describe as "the fixed header size," not memorize precisely) marks where the fixed header ends and the **dynamic data region** begins. From `HEADER_OFFSET` onward, the array holds, index-for-index in parallel with `TView.data`:

- DOM node references (or component/directive instances) for each element/text/container in the template, in creation order.
- Binding values — the previous values of interpolations/property bindings, used for dirty-checking (`bindingupdated / @property` compare against the stored previous value before touching the DOM).
- Pipe instances.
- `LContainer`s for structural directives (a special array-of-arrays structure holding the list of embedded `LView`s currently projected into that container, plus the anchor comment node).
- Directive instances co-located with their host element's slot region.

So a node's "identity" at runtime is really just **an integer index** into this array — that's the `TNode.index` you'll see threaded through instruction calls (`@advance(2)` literally moves an internal cursor, `TView`'s "selected index," forward by 2 slots).

2.4 TNode — per-node static shape

Every element, text node, container, ICU, projection placeholder, etc. has a corresponding `TNode` living in `TView.data` at the same index as its live counterpart in `LView`. A `TNode` holds:

- `type` (Element, Text, Container, ElementContainer/`ng-container`, Icu, Projection).
- `index` — its own slot index (redundant but convenient).
- `attrs` — static attributes compiled to a flat array.
- `localNames` — template reference variables (`#foo`) declared on this node.
- `inputs / outputs` — property/event binding metadata, per directive.
- `directiveStart / directiveEnd` — the slice of directive-def indices matched to this node.
- `parent, child, next, prev` — the **static tree shape**, a linked structure mirroring the DOM/template hierarchy, used to walk the tree without re-deriving structure from the live DOM.
- `providerIndexes` — for the node injector.

The static tree (`TNode.parent/child/next`) plus the parallel `LView` data array is how Angular reconstructs "what does this component's DOM look like" without ever needing to query the real DOM — the DOM is a *side effect* the renderer produces, not the source of truth Angular reasons from.

2.5 The instruction-set rendering model

A compiled template is, at its core, two functions: a **create block** and an **update block**, gated by `if (rf & RenderFlags.Create) { ... } if (rf & RenderFlags.Update) { ... }`. The runtime calls this single function twice per lifecycle-relevant pass conceptually (though usually merged), passing a `RenderFlags` bitmask so the same function handles both "first time, build the nodes" and "every subsequent CD run, just refresh bindings."

Illustrative compiled shape:

```
function ItemComponent_Template(rf: RenderFlags, ctx: ItemComponent) {
  if (rf & 1 /* Create */) {
    @elementStart(0, 'div', 0);
    @text(1);
    @elementEnd();
  }
  if (rf & 2 /* Update */) {
    @advance(1);
    @textInterpolate1('Qty: ', ctx.qty, '');
  }
}
```

Every instruction is a plain exported function in `@angular/core` that:

1. Reads the "currently selected `LView`/`TView`" from module-level runtime globals (`getLView()`, `getTView()` — Angular's instruction execution is *not* re-entrant-safe by accident; it relies on a single-threaded, stack-based "currently active view" pointer that instructions push/pop as they descend into child views/components).
2. On the create pass, allocates the `TNode`, calls into the active `Renderer2` to actually create the DOM node (`renderer.createElement(...)`), and stores the DOM ref into the `LView` slot.
3. On the update pass, compares the new binding value against the previous one stored in the same slot (`bindingUpdated0..4 / bindingUpdated(lview, index, value)`), and only if changed, calls the renderer to apply it (`renderer.setProperty`, `renderer.setAttribute`, etc.) and writes the new value back into the slot.

This is why Angular's change detection, at the lowest level, is **not** "diff the whole tree" — it's "walk a known linear sequence of binding slots and skip identical values," an $O(\text{bindings})$ operation with no tree-diffing at all. The perceived "tree walk" is really the runtime advancing the `TView`'s node cursor and descending into child component `LViews` that are reachable via `LView.CHILD_HEAD`/`TView.components`.

2.6 The renderer abstraction: `Renderer2` vs `Renderer3`

`Renderer2` is the public, stable, injectable class (`@angular/core`) that application code and directives can inject to imperatively touch the DOM in a renderer-agnostic way (`renderer.setStyle(e1, 'color', 'red')`, `renderer.listen(...)`). It exists so user code never calls `e1.style.color = 'red'` directly, which would break server-side rendering (no real DOM) and would bypass Angular's sanitization/testability hooks.

`Renderer3` (internal, not exported publicly, defined in `interfaces/renderer.ts`) is the low-level interface the *instruction set itself* calls into. It's a plain object of functions — `createElement`, `createText`, `createComment`, `appendChild`, `insertBefore`, `removeChild`, `setAttribute`, `setProperty`, `setStyle`, `addClass`, `listen`, `parentNode`, `nextSibling` — deliberately shaped close to raw DOM API calls but backend-agnostic. `Renderer2` instances used by application code are, in the default browser configuration, a thin object that ultimately delegates to the same underlying primitives.

A `RendererFactory2` (also injectable, overridable) produces renderer instances per-view/per-root-element; `platform-browser` supplies `DomRendererFactory2`, which creates renderers that talk to the real `document`; `platform-server` (`@angular/platform-server`) supplies a factory whose renderers build up a serialized string/DOM-emulation tree instead (backed by `domino` / a DOM-emulation layer historically, or the real Node DOM shim depending on version); Angular's own test renderer used by `TestBed` can be a no-op/counting renderer for unit tests that don't need real DOM assertions.

Because every single DOM-touching instruction goes through this interface and never touches `document` / `window` directly, the **exact same compiled template function** runs correctly in a browser tab, in a Node.js SSR process, or inside a headless test — the compiler output is identical; only the injected renderer differs. This is the practical payoff of the compiler/runtime split.

2.7 The LView tree — how a running app is actually shaped

A running Angular application is a tree of `LViews`, not a tree of components in the OOP sense. Precisely:

- The root `LView` corresponds to the bootstrapped component's host view.
- Each component instance owns its own `LView` (with `CONTEXT` = the component class instance), linked to its parent via `PARENT` / `T_HOST`.
- Each structural directive instantiation (`*ngIf`, `*ngFor`, `<ng-template>` usage, dynamic `viewContainerRef.createEmbeddedView`) produces an `LContainer` sitting at a slot in the parent `LView`; the `LContainer` in turn holds a list of embedded `LViews` (its "views"), each sharing the `<ng-template>`'s single `TView`.
- Content-projected views retain a `DECLARATION_VIEW` pointer back to where they were *lexically declared* (for DI/context resolution) distinct from where they are *inserted* (`T_HOST` / parent) — this split is precisely what makes `<ng-template>` + `ngTemplateOutlet` / portals work: a template can be declared in one component and rendered inside a completely different component's DOM location while still resolving `this` and DI against its declaration site.

Change detection (`ApplicationRef.tick()` → `refreshView()` internally, or per-view via `ChangeDetectorRef`) is fundamentally a **depth-first walk of this LView tree**, refreshing each `LView`'s bindings via its `TView`'s update block, then recursing into `LContainer`s and child component `LViews` — pruned by `CheckAlways` / `OnPush` / dirty flags on `LView.FLAGS`.

2.8 NgModules, standalone components, and the runtime

This is a frequent point of confusion: **the runtime does not know or care about NgModules as a bootstrapping concept beyond dependency-injection scoping and compilation-time selector resolution**. Both NgModule-declared and standalone components compile to the exact same `ɵcmp` definition shape (`ɵdefineComponent`), consumed by the exact same instruction set.

- An `@NgModule` compiles to `ɵmod (ɵdefineNgModule) + ɵinj (ɵdefineInjector)` static properties —

these exist purely to describe **DI scoping** (which providers are visible where) and, historically, **template compilation scope** (which directives/pipes/components are visible to templates compiled "inside" that module, resolved via `NgModule.declarations / imports`).

- A standalone component compiles to the same `@cmp`, but its template-scope resolution (`directiveDefs / pipeDefs` on the component def) is computed directly from the component's own `imports` array at compile time — no module indirection needed. `@defineComponent`'s `dependencies` field is populated either way; NgModules just used to be the *only* mechanism to compute that list indirectly through `declarations`.
- Bootstrapping (`bootstrapApplication` for standalone vs `platformBrowserDynamic().bootstrapModule` for NgModule-based) differs only in **how the root injector and initial providers are assembled** — `bootstrapApplication` builds an `EnvironmentInjector` directly from the array of providers/ `importProvidersFrom(...)` calls you supply, whereas `bootstrapModule` walks the NgModule graph to build the same kind of `EnvironmentInjector`. Once bootstrap completes, both produce an `ApplicationRef` driving the same `LView`-tree-based rendering runtime. There is no separate "standalone runtime."

2.9 Platform abstraction

`@angular/core` defines `PlatformRef` and the concept of a **platform injector** — a small, long-lived injector created once per process (`createPlatform / platformCore`) that sits above every application's root injector, providing platform-specific low-level tokens (e.g., `DOCUMENT`, `PLATFORM_ID`, initial platform providers, exception handling defaults).

- `platform-browser` (`platformBrowser() / platformBrowserDynamic()`) provides the browser `DOCUMENT` (the real `window.document`), a `DomRendererFactory2` backed by real DOM APIs, `BrowserModule`'s meta-services (Title, Meta, HammerGesture support, etc.), and `PLATFORM_ID = 'browser'`.
- `platform-server` (`@angular/platform-server`, powering Angular Universal / the Node SSR builder) provides an emulated `DOCUMENT`, a renderer factory that produces serializable markup instead of live browser DOM, `PLATFORM_ID = 'server'`, and hooks for `TransferState`/hydration bookkeeping.
- `platform-browser-dynamic` additionally wires up the JIT compiler for cases where templates are compiled at runtime rather than at build time (largely legacy now that the CLI defaults to AOT everywhere, including `ng serve`).

Because the instruction set and `LView / TView` machinery are platform-agnostic — they only ever call through `Renderer3` and read `DOCUMENT` via DI — the same compiled component code runs unmodified across all of these; platform packages only ever swap out the **environment the runtime is handed**, never the runtime itself. Hydration (`provideClientHydration()`) is a further refinement on top of this: the server-rendered DOM is *reused* (nodes matched up and claimed) rather than destroyed and rebuilt, which required teaching the browser-side renderer to run in a "claim mode" against `Renderer3`'s create-instructions instead of always calling `createElement`.

3. Code Examples

3.1 Illustrative shape of an LView (simplified, not literal source)

```
// This is a conceptual illustration of what @angular/core's internal
```

```

// LView array roughly holds – NOT the literal current source, which lives
// in packages/core/src/render3/interfaces/view.ts and evolves across versions.

const HEADER_OFFSET = 20; // first index of per-node dynamic data

// Fixed header slot indices (illustrative constants)
const enum LViewSlot {
  TVIEW = 0,
  FLAGS = 1,
  PARENT = 2,
  NEXT = 3,
  T_HOST = 5,
  CLEANUP = 6,
  CONTEXT = 7,
  INJECTOR = 8,
  ENVIRONMENT = 9,
  RENDERER = 10,
  CHILD_HEAD = 11,
  CHILD_TAIL = 12,
  DECLARATION_VIEW = 13,
  QUERIES = 14,
  // ...more bookkeeping slots up to HEADER_OFFSET
}

// A concrete LView for a single <app-item [qty]="5"> instance might look like:
const exampleLView: any[] = [
  /* TVIEW */      tviewForAppItem,
  /* FLAGS */      0b0000_0001_0100, // e.g. CreationMode | Attached | Dirty bits
  /* PARENT */     parentLView,
  /* NEXT */       null,
  /* (reserved) */ 0,
  /* T_HOST */     appItemTNode,
  /* CLEANUP */    null,
  /* CONTEXT */    new AppItemComponent(), // the actual component instance
  /* INJECTOR */   nodeInjectorForThisView,
  /* ENVIRONMENT */ sharedEnvironment,      // renderer factory, sanitizer...
  /* RENDERER */   domRendererInstance,
  /* CHILD_HEAD */ null,
  /* CHILD_TAIL */ null,
  /* DECLARATION_VIEW */ parentLView,
  /* QUERIES */     null,
  // ...remaining header slots omitted...

  // ---- HEADER_OFFSET (index 20) starts the per-node dynamic region ----
  /* [20] div element */      divElementNode,      // RNode (real/emulated DOM element)
  /* [21] text node */        textNode,            // RNode
  /* [22] prev binding val */  5,                  // previous value of `qty` interpolation
  /* [23] LContainer for *ngIf, if any child structural directive exists */ undefined,
];

```

3.2 Illustrative TView shape paired with the LView above

```

// Conceptual shape of the shared TView for AppItemComponent's template.
// One TView instance is created on the *first* AppItemComponent instantiation
// and then reused by every subsequent instance's LView[TVIEW] slot.

interface IllustrativeTNode {
  index: number;
  type: 'Element' | 'Text' | 'Container' | 'ElementContainer' | 'Icu' | 'Projection';
  attrs: (string | number)[] | null;
  directiveStart: number;
  directiveEnd: number;
  parent: IllustrativeTNode | null;
  child: IllustrativeTNode | null;
  next: IllustrativeTNode | null;
}

const tviewForAppItem = {
  type: 'Component' as const,
  firstCreatePass: false, // flips to false after the first instance is built
  bindingStartIndex: 22, // index 22 in LView is where bindings begin
  data: [
    /* ...header-aligned placeholders... */
    /* [20] */ { index: 20, type: 'Element', attrs: ['class', 'item'], directiveStart: -1,
    directiveEnd: -1, parent: null, child: null, next: null } as IllustrativeTNode,
    /* [21] */ { index: 21, type: 'Text', attrs: null, directiveStart: -1, directiveEnd: -1,
    parent: null, child: null, next: null } as IllustrativeTNode,
  ],
  template: function AppItemComponent_Template(rf: number, ctx: any) {
    if (rf & 1) {
      // @elementStart(20, 'div', 0); @text(21); @elementEnd();
    }
    if (rf & 2) {
      // @advance(1); @textInterpolate1('Qty: ', ctx.qty, '');
    }
  },
  cleanup: null,
  contentHooks: null,
  viewHooks: null,
  destroyHooks: null,
  components: null, // indices of any child component nodes
};

```

3.3 A minimal custom `Renderer2` (illustrating the abstraction boundary)

```

import {
  Injectable, RendererFactory2, Renderer2, RendererType2,
} from '@angular/core';

/**
 * A minimal Renderer2 implementation that logs every DOM mutation before
 * delegating to a real target renderer. Demonstrates that Renderer2 is just
 * an interface: anything satisfying it can sit between instructions and the

```

```

* real rendering target (browser DOM, SSR buffer, a canvas, a native widget
* tree, etc.) – which is precisely how platform-server and testing renderers
* slot in without the compiler/instruction layer ever changing.
*/
@Injectable()
export class LoggingRendererFactory implements RendererFactory2 {
  constructor(private readonly delegateFactory: RendererFactory2) {}

  createRenderer(hostElement: any, type: RendererType2 | null): Renderer2 {
    const delegate = this.delegateFactory.createRenderer(hostElement, type);
    return new LoggingRenderer(delegate);
  }
}

class LoggingRenderer implements Renderer2 {
  constructor(private readonly delegate: Renderer2) {}

  get data(): { [key: string]: any } {
    return this.delegate.data;
  }

  destroy(): void {
    this.delegate.destroy();
  }

  createElement(name: string, namespace?: string | null) {
    console.log('[render] createElement', name);
    return this.delegate.createElement(name, namespace);
  }

  createComment(value: string) {
    return this.delegate.createComment(value);
  }

  createText(value: string) {
    return this.delegate.createText(value);
  }

  appendChild(parent: any, newChild: any): void {
    console.log('[render] appendChild');
    this.delegate.appendChild(parent, newChild);
  }

  insertBefore(parent: any, newChild: any, refChild: any, isMove?: boolean): void {
    this.delegate.insertBefore(parent, newChild, refChild, isMove);
  }

  removeChild(parent: any, oldChild: any, isHostElement?: boolean): void {
    console.log('[render] removeChild');
    this.delegate.removeChild(parent, oldChild, isHostElement);
  }

  selectRootElement(selectorOrNode: string | any, preserveContent?: boolean) {

```

```

    return this.delegate.selectRootElement(selectorOrNode, preserveContent);
}

parentNode(node: any) {
    return this.delegate.parentNode(node);
}

nextSibling(node: any) {
    return this.delegate.nextSibling(node);
}

setAttribute(el: any, name: string, value: string, namespace?: string | null): void {
    console.log('[render] setAttribute', name, '=', value);
    this.delegate.setAttribute(el, name, value, namespace);
}

removeAttribute(el: any, name: string, namespace?: string | null): void {
    this.delegate.removeAttribute(el, name, namespace);
}

addClass(el: any, name: string): void {
    this.delegate.addClass(el, name);
}

removeClass(el: any, name: string): void {
    this.delegate.removeClass(el, name);
}

setStyle(el: any, style: string, value: any, flags?: number): void {
    this.delegate.setStyle(el, style, value, flags);
}

removeStyle(el: any, style: string, flags?: number): void {
    this.delegate.removeStyle(el, style, flags);
}

setProperty(el: any, name: string, value: any): void {
    console.log('[render] setProperty', name, '=', value);
    this.delegate.setProperty(el, name, value);
}

setValue(node: any, value: string): void {
    this.delegate.setValue(node, value);
}

listen(target: any, eventName: string, callback: (event: any) => boolean | void) {
    console.log('[render] listen', eventName);
    return this.delegate.listen(target, eventName, callback);
}
}

```

```

// Registered via a factory provider so it wraps whichever platform-supplied
// RendererFactory2 (DomRendererFactory2 in the browser, the server variant

```

```
// under SSR) is already configured – the same wrapping class works in both.
export function loggingRendererFactoryProvider(existing: RendererFactory2) {
  return new LoggingRendererFactory(existing);
}
```

4. Internal Working

4.1 The creation pass, step by step

1. **Bootstrap** (`bootstrapApplication / bootstrapModule`) creates the root `EnvironmentInjector`, resolves the root component, and calls into `createRootComponent`, which allocates the **root** `LView` and its `TView` (or reuses a cached one if this component type has already been bootstrapped before, which matters for `TestBed` re-runs).
2. Angular calls the component's compiled **template function with** `RenderFlags.Create`. The runtime maintains a module-scoped "current view" pointer (`setCurrentTNode / enterView / leaveView` internally) so that instruction functions, which take no `LView` argument for brevity, always know which view/TView they're mutating.
3. For each `ɵɵelementStart(index, tagName, attrsIndex)` call:
 - If `tview.firstCreatePass` is true, the runtime **creates a new** `TNode`, resolves which directives match this node (by walking the compiled `directiveRegistry` and comparing selectors — this is where content-scope resolution from `imports / declarations` actually gets consulted), and appends it into `TView.data` and the static tree (`parent / child / next` pointers).
 - Unconditionally, the runtime calls `renderer.createElement(tagName)` and stores the resulting node reference into `LView[index]`.
 - Static attributes recorded on the `TNode` are applied via `renderer.setAttribute`.
4. `ɵɵelementEnd()` pops the current element off an internal "creation stack" so nested `elementStart` calls build the correct parent/child DOM relationships via `renderer.appendChild`.
5. For directive-bearing nodes, the runtime **instantiates directive/component instances** (`getNodeInjectable`), running constructor DI resolution against the node injector chain (which walks up `LView.PARENT / T_HOST` and the environment injector hierarchy) and stores each instance into its own `LView` slot.
6. If the node is itself a component host, the runtime **recursively creates that child component's own** `LView / TView` (creating its `TView` on first encounter, reusing thereafter) and links it as a child view — this is the recursive step that builds the `LView` tree depth-first as creation proceeds.
7. Once the whole create pass for a `TView` finishes for the first time, `firstCreatePass` flips to `false` permanently for that `TView` — every later instance of the same component skips all the `TNode`-building/directive-matching work and only allocates a fresh `LView` array plus fresh directive instances.

4.2 The update pass, step by step

1. Triggered by `ApplicationRef.tick()` (whole-app CD), a zone `onMicrotaskEmpty` event (in zone.js-based apps), a signal write scheduling a notification (in zoneless/signals-driven change detection), or an explicit `ChangeDetectorRef.detectChanges()`.
2. The runtime calls `refreshView(tview, lview, ...)`, which re-invokes the same template function

with `RenderFlags.Update`.

- Each `ɵɵadvance(n)` moves the "selected index" cursor forward by `n` slots — this is what lets `ɵɵproperty` / `ɵɵtextInterpolateN` calls not need an explicit index argument; they operate on "whatever the cursor currently points at."
- Each binding instruction (`ɵɵproperty(name, value)`, `ɵɵtextInterpolate1(...)`) calls `bindingUpdated(lview, bindingIndex, value)`, which does a **reference/=== comparison** against the value already stored at that slot. If unchanged, the instruction returns early — no renderer call happens at all. If changed, it stores the new value and calls the appropriate `renderer.setProperty / setAttribute` /etc.
- After a view's own bindings are refreshed, the runtime walks `TView.components` (child component node indices) and recurses `refreshView` into each child's `LView` — **unless** that child is `OnPush` and not marked dirty (`LView.FLAGS & Dirty` not set and no `markForCheck()` /signal-input change occurred), in which case the whole subtree is skipped. This flag check, not a "compare virtual DOM trees" step, is the entirety of Angular's OnPush optimization at the runtime level.
- `LContainer`s (structural-directive-created view lists) are walked similarly: each contained embedded `LView` gets `refreshView`'d in insertion order.

4.3 How platform abstraction makes this portable

Nothing in sections 4.1/4.2 references `document`, `window`, or any browser global directly — every DOM-shaped operation is a call through `LView[RENDERER]` (a `Renderer3`), obtained originally from the `RendererFactory2` that was provided by whichever platform module (`platform-browser` vs `platform-server`) configured the root injector. `DOCUMENT` itself is an injection token, not a global reference, specifically so `platform-server` can bind it to an emulated document.

This is why:

- Running the **same** compiled `AppComponent` output under `bootstrapApplication` (browser) vs. `renderApplication` (server, from `@angular/platform-server`) produces correct output in both — the template function, `TView`, instruction set are byte-identical; only the injected `RENDERER` / `DOCUMENT` differ.
- `TestBed`-based unit tests can run without a real browser (e.g., under Node with `jsdom` / `domino`, or headless Chrome via Karma) because the DOM interaction always funnels through the same swappable seam.
- Hydration works by having the server-rendered markup double as the "expected DOM," and the browser-side renderer, when hydration is enabled, runs in a mode where creation instructions **look up and claim existing nodes** (matching by position/comment-node markers Angular injects during SSR) instead of calling `createElement`, before falling back to normal creation for anything unmatched.

5. Edge Cases & Gotchas

- `TView.firstCreatePass` **bugs are subtle and rare but real**: any logic that behaves differently based on runtime *state* rather than static template shape and gets baked in during the first pass can produce stale behavior for later instances. This is almost entirely an Angular-core-internals concern, not application code — but it explains why certain Ivy regressions historically only reproduced with 2+ instances of a component on screen simultaneously.

- **LView reuse across TestBed reset:** `TestBed.resetTestingModule()` invalidates a component's `TView`/def cache tied to that testing module's compilation scope; failing to reset between tests that redefine the same component's template (`TestBed.overrideComponent()`) can leak stale `TView` state, especially under `--test=isolated` differences across CLI versions.
- **OnPush + direct DOM/array mutation looks "broken" but is architecturally correct:** because dirty-checking is a flag on `LView.FLAGS`, not a deep-equality check, mutating an `@Input()` object's internals in place never sets that flag, so `refreshView` legitimately skips the subtree — this is the direct, mechanical explanation for "OnPush didn't detect my array push," not a bug.
- **ɵadvance cursor mismatches from hand-written instruction code:** if you ever hand-write instruction-level code (rare, but happens in advanced custom-renderer or meta-programming scenarios, or when reading compiler-generated diffs during debugging), an incorrect `ɵadvance` count desyncs the cursor from the intended `TNode`, causing bindings to apply to the wrong element — a category of bug that literally cannot occur when you only write templates, since the compiler always emits correct offsets.
- **Content projection and DECLARATION_VIEW vs. insertion parent confusion:** a `<ng-template>` passed via `ngTemplateOutlet`/CDK portals resolves `this`/DI against where it was *declared*, but renders into the DOM location where it's *inserted* — forgetting this split is a common source of "why can this template see that service/component context" confusion in advanced projection scenarios.
- **Renderer2 vs. direct ElementRef.nativeElement mutation:** bypassing `Renderer2` to mutate `nativeElement` directly works fine in the browser but silently breaks (or requires shims) under SSR, since there's no real DOM to mutate — a very common lint-rule-worthy gotcha (`@angular-eslint` flags direct native element access for exactly this reason).
- **Standalone component imports don't create the isolation people expect:** a standalone component's template dependency resolution is per-component, but **DI is still governed by EnvironmentInjector hierarchy**, not by `imports` — importing a component doesn't scope its providers to "only this component," a frequent source of confusion when migrating off `NgModules`.
- **Shared embedded-view TView means per-iteration state must live in the LView, never the TView:** because every `*ngFor` row shares one `TView`, any attempt (in low-level or meta-programming code) to stash "this iteration's value" on the `TView` rather than the `LView`/context object would corrupt every row — a good check for whether you truly understand the `TView/LView` split.

6. Interview Questions & Answers

Q1. What is the difference between the Angular compiler and the Angular runtime, at a package level?

A: The compiler (`@angular/compiler`, `@angular/compiler-cli`) turns templates and decorator metadata into plain TypeScript/JavaScript — specifically calls into a stable instruction set, plus static `ɵcmp / ɵmod / ɵinj / ɵfac` definitions. The runtime (`@angular/core`) is the library that *implements and executes* those instructions against `LView/TView` data structures. AOT does compilation at build time; the runtime is what actually runs in the browser/server regardless.

Q2. What is an LView, in one sentence, and why is it a plain array rather than a class instance?

A: An `LView` is the mutable, per-instance runtime state of one component or embedded view instance — a flat array holding a fixed "header" of bookkeeping slots (`TView` pointer, parent, context, renderer, flags, etc.) followed by a dynamic per-node region storing DOM refs, directive instances, and previous binding values. It's a plain array rather than a class instance because monomorphic array access is faster and more predictably

optimized by JS engines than property access on many differently-shaped class instances, and it avoids allocating a full object graph per node.

Q3. What is a `TView`, and how does it relate to an `LView`?

A: `TView` is the static, shared "template blueprint" for a given component type or embedded-view template — created once (on first instantiation) and cached on the component definition, then reused by every `LView` instance of that type. It holds `TNode`s (per-node static shape/attributes/directive matches), the compiled template function, and hook/query metadata. Every `LView` has a pointer back to its `TView` at a fixed header slot; two component instances share the same `TView` but have distinct `LViews`.

Interviewer intent: This checks whether the candidate actually understands Ivy's "compile once, run many" model, versus just repeating buzzwords. A strong answer explicitly says *shared* and *cached* and ties it to performance (avoiding repeated directive-matching/`TNode`-building per instance).

Q4. What is `HEADER_OFFSET` and why does it exist?

A: It's the fixed number of header slots at the start of every `LView` (and the parallel `TView.data`) reserved for framework bookkeeping (`TView` pointer, flags, parent, context, injector, renderer, child-view links, etc.) before the per-node dynamic data region begins. It exists so instruction code can address "the Nth template node" using a small integer offset from a known constant, without every `LView` needing a different layout depending on how much bookkeeping it happens to carry.

Q5. Explain what happens, mechanically, when Angular runs change detection on an `OnPush` component that hasn't changed.

A: `refreshView` is invoked on that component's `LView`. Before descending into its subtree, the runtime checks `LView.FLAGS` for a dirty/`RefreshView` bit. For an `OnPush` component, that bit is only set if `markForCheck()` was called, a signal read inside its template changed, or (historically) one of its `@Input()` references changed identity. If none of those occurred, the flag is clear, and the runtime skips refreshing that `LView`'s bindings and skips recursing into its children entirely — it's a flag check, not a value comparison against the DOM or a virtual tree.

Q6. What is the difference between `Renderer2` and `Renderer3`?

A: `Renderer2` is the stable, public, injectable abstract class application/directive code can use to perform renderer-agnostic DOM operations. `Renderer3` is the internal, lower-level functional interface (a plain object of functions like `createElement`, `setProperty`, `listen`) that the compiled instruction set itself calls into during create/update passes. `Renderer2` instances are effectively thin, publicly-typed wrappers around the same underlying primitives that back `Renderer3` in a given platform's `RendererFactory2`.

Interviewer intent: Many mid-level candidates only know `Renderer2` exists as "the thing you inject instead of touching `nativeElement`." This question checks whether they understand there's a second, lower internal layer that the instruction set itself depends on — i.e., whether they've looked past the public API surface at all.

Q7. How does Angular support server-side rendering without the compiler emitting different code for server vs. browser?

A: Because the compiler's output only ever calls the instruction set, and the instruction set only ever talks to the DOM through the injected `Renderer3`/`DOCUMENT` tokens, the compiled template function is platform-agnostic. `platform-browser` binds those tokens to real DOM APIs; `platform-server` (`@angular/platform-server`) binds them to a server-side DOM emulation/serialization layer instead. Swapping the platform package swaps the environment the same compiled code runs against — no separate server-specific compiled output is needed.

Q8. What actually gets shared vs. duplicated when you render a list of 1,000 items with `*ngFor`?

A: All 1,000 rows share **one** `TView` (the embedded view's template compiled once from the `<ng-template>` `*ngFor` desugars to) and its `TNode` tree/static attributes/directive matches. Each row gets its **own** `LView` — its own array of DOM node references, its own previous-binding-value slots, its own directive/component instances if the row template instantiates any. This is why `*ngFor` scales reasonably well: the expensive, one-time structural analysis happens once, and per-row cost is "allocate an array and run the update block."

Q9. How do NgModule-declared components and standalone components differ at the compiled-output level?

A: Both compile to the same `ɵcmp` shape via `ɵdefineComponent`, consumed by the identical runtime instruction set — there is no separate "standalone runtime." The difference is only in how each component's `dependencies` (which other directives/pipes/components its template can reference) get computed: for NgModule components, it's derived indirectly through the module's `declarations` / `imports` graph; for standalone components, it's computed directly from the component's own `imports` array. DI scoping differs similarly — module-based apps build the root `EnvironmentInjector` by walking the NgModule graph (`ɵmod` / `ɵinj`), while `bootstrapApplication` builds it directly from a flat providers array.

Q10. Why does Angular re-invoke the same template function for both creation and update, gated by a `RenderFlags` bitmask, rather than compiling two separate functions?

A: It keeps the create and update logic for a given node co-located in the compiler output (easier for the compiler to emit correctly and for engineers to read/debug), and lets the runtime reuse the exact same "walk to node N" traversal logic (via `ɵadvance`) for both phases rather than maintaining two parallel index-tracking schemes. The `rf & 1` / `rf & 2` branches simply let dead-code elimination and V8's JIT skip irrelevant branches cheaply at runtime.

Q11. What is an `LContainer` and how does it relate to `*ngIf` / `*ngFor`?

A: An `LContainer` is a specialized array-like structure occupying a slot in a parent `LView`, representing a **view container** — the runtime location where zero or more embedded `LViews` are inserted/removed dynamically. Structural directives like `*ngIf` / `*ngFor` are sugar over `<ng-template>` plus a `ViewContainerRef`; at runtime, `NgIf` calls `viewContainerRef.createEmbeddedView / clear()` against the `LContainer` at its host node's slot, inserting or removing `LView`s (all sharing the `<ng-template>`'s single `TView`) as the bound condition/collection changes.

Interviewer intent: Tests whether the candidate can connect the "directive as a class using `TemplateRef / ViewContainerRef`" API-level understanding to the concrete runtime structure (`LContainer`) it manipulates — a common gap between "I've used `*ngIf`" and "I understand what `*ngIf` does under the hood."

Q12. During the first-ever creation pass of a component's `TView`, what specifically gets computed that is later skipped for subsequent instances?

A: `TNode` construction for every element/text/container node in the template (type, static attrs, static tree parent/child/next pointers), directive-matching against each node's selector-matchable attributes/tag (resolving which `directiveDefs` apply, populating `directiveStart` / `directiveEnd`), binding-index/expando-slot layout assignment for host bindings, and query/hook registration (`contentHooks` / `viewHooks` / `cleanup` arrays). All of this is gated behind `tView.firstCreatePass`; every later instance skips straight to allocating a fresh `LView` and running the already-known creation/update instruction sequence.

Q13. Why is the phrase "Angular does virtual DOM diffing" incorrect, and what does it actually do instead?

A: Angular never builds an intermediate virtual-tree representation of the whole component that it diffs against a previous snapshot. Instead, the compiler emits a fixed, ordered sequence of binding instructions per template; at runtime, each instruction independently compares its *own* new value against the single previous value cached in its own `LView` slot (`bindingUpdated`), and only issues a targeted renderer call if that one value changed. There's no tree comparison step and no reconciliation/keying algorithm for elements themselves (only `*ngFor`'s `trackBy` plays a role resembling "keys," and only for view reuse/reordering, not for a diff algorithm).

Q14. How does hydration reuse this architecture rather than requiring a wholly separate mechanism?

A: Hydration doesn't change the instruction set or `LView`/`TView` model at all — it changes what the create-pass instructions *do* when `provideClientHydration()` is active and matching server-rendered markup exists. Instead of unconditionally calling `renderer.createElement`, the runtime's node-creation path attempts to locate and "claim" an existing DOM node (using comment-node markers Angular embedded during SSR to align template structure with serialized markup), populating the same `LView` slots with references to the *existing* nodes rather than newly created ones, and falling back to normal creation for any unmatched node. The update pass and the rest of change detection proceed completely unmodified afterward.

7. Quick Revision Cheat Sheet

- **Compiler vs runtime:** compiler turns templates/decorators into instruction-set calls + static defs (`@cmp / @mod / @inj / @fac`); runtime (`@angular/core`) implements/executes those instructions against `LView`/`TView`.
- **TView** = static, shared-per-type template blueprint (`TNodes`, hooks, template fn, query defs); created once, cached, reused by every instance.
- **LView** = mutable, per-instance array: fixed header (`TView` ptr, flags, parent, context, injector, renderer, child-view links) + dynamic region (DOM refs, directive instances, previous binding values).
- **HEADER_OFFSET**: boundary index where the fixed bookkeeping header ends and per-node dynamic data begins.
- **TNode**: static per-node metadata (type, attrs, directive matches, static tree parent/child/next) living in `TView.data`, index-aligned with the corresponding `LView` slot.
- **Instruction set:** small, stable, exported `@angular/core` functions (`@elementStart`, `@property`, `@advance`, ...) that the compiler emits calls to; gated by `RenderFlags.Create / Update` in one shared template function per component.
- **Change detection mechanics:** `bindingUpdated` compares new vs. previously-cached slot value; only issues a renderer call on change — no virtual-DOM diffing, no tree comparison.
- **Renderer2**: public, injectable, renderer-agnostic API for app/directive code. **Renderer3**: internal functional interface the instruction set itself calls; **Renderer2** is a thin wrapper over the same primitives.
- **LView tree:** root `LView` → component `LViews` (via `T_HOST / PARENT`) → `LContainers` (view containers for structural directives) → embedded `LViews`, all sharing their defining `<ng-template>`'s single `TView`.
- **firstCreatePass**: expensive `TNode`-building/directive-matching/query-registration work happens once per `TView`; later instances skip straight to cheap `LView` allocation + instruction execution.

- **NgModules vs standalone:** both compile to identical `ɵcomp` /instruction-set shape; they differ only in how template-dependency lists and DI scoping/root injector assembly are computed (module graph walk vs. direct `imports` /providers array).
- **Platform abstraction:** `platform-browser` binds `DOCUMENT` /renderer factory to real DOM; `platform-server` binds them to an emulated/serializable DOM — same compiled instructions run against either unmodified.
- **Hydration:** reuses the identical creation instructions in a "claim existing node" mode instead of always creating fresh nodes; no separate rendering pipeline.

Created By - Durgesh Singh